

Phần 4 - Quality Attributes

Mục lục

4.1 Tổng quan Phần 4: Quality Attributes	3
Câu hỏi mở đầu	3
Quality Attribute là gì?	3
Nếu bạn đến từ Data Science	4
Phần 4 sẽ dạy gì?	4
Bài 4.2: Functional vs Non-functional Requirements	4
Bài 4.3: Identifying Architecture Characteristics	4
Bài 4.4: Component-Based Thinking	4
Bảng QA phổ biến	4
Operational QA: xảy ra khi hệ thống chạy	4
Structural QA: ảnh hưởng cách code và team phát triển	5
Cross-cutting QA: cắt ngang nhiều phần	5
Vì sao không thể chọn tất cả?	6
Accuracy vs Latency	6
Freshness vs Cost	6
Privacy vs Observability	6
Simplicity vs Scalability	6
Explainability vs Performance	6
Cách operationalize QA	6
Ví dụ: chọn architecture cho churn prediction	7
Option 1: Batch inference mỗi đêm	7
Option 2: Online inference khi CRM mở profile	7
Architect's responsibility	8
Self-check	8
Cách đọc phần này	8
4.2 Functional vs Non-functional Requirements	9
Nếu bạn đến từ Data Science	9
Câu hỏi mở đầu	9
Định nghĩa formal	9
Functional Requirement (FR)	9
Non-functional Requirement (NFR)	10
Architecture Characteristic (Mark Richards)	10
Implicit vs Explicit Characteristics	10
Explicit	10
Implicit	10
"Architecture is the X-ilities"	11

Ví dụ thực hành: từ FR □ NFR □ Architecture	11
Use case: Hệ booking phim cinema	11
NFR khó hơn FR	11
Khó measure	11
Khó test	12
Khó negotiate với business	12
Conflict nhau	12
Sai lầm thường gặp	12
Sai lầm 1: Bỏ qua NFR trong design phase	12
Sai lầm 2: NFR không measurable	12
Sai lầm 3: Treat tất cả NFR như nhau	12
Sai lầm 4: NFR không liên kết với business value	12
Tóm tắt	12
4.3 Xác định Architecture Characteristics	13
Nếu bạn đến từ Data Science	13
Vì sao “không quá 7”?	13
Pipeline identify	13
Bước 1: Đọc requirement document	13
Bước 2: Stakeholder interview	13
Bước 3: Implicit characteristics	14
Bước 4: Priority với business	14
Bước 5: Operationalize	14
Hierarchy của Characteristics	15
Operational (Runtime characteristics)	15
Structural (Development characteristics)	15
Cross-cutting	15
CQA Worksheet (Quality Attribute Workshop simplified)	16
Sai lầm thường gặp	16
Sai lầm 1: “Mọi thứ đều quan trọng”	16
Sai lầm 2: QA không measurable	16
Sai lầm 3: Implicit không document	16
Sai lầm 4: QA conflict không resolve	16
Sai lầm 5: Không revisit	17
Tóm tắt	17
4.4 Component-Based Thinking	18
Nếu bạn đến từ Data Science	18
Component là gì?	18
Component vs Module, phân biệt	18
Identify Component	18
1. Entity-based (theo domain entity)	19
2. Workflow-based (theo use case)	19
Combined approach	19
Component Cohesion Principles (Robert C. Martin)	19
REP, Reuse/Release Equivalence Principle	19
CCP, Common Closure Principle	20
CRP, Common Reuse Principle	20
Tension giữa 3 principles	20

Component Coupling Principles	20
ADP, Acyclic Dependencies Principle	20
SDP, Stable Dependencies Principle	21
SAP, Stable Abstractions Principle	21
Main Sequence	22
Sai lầm thường gặp	22
Sai lầm 1: Component quá nhỏ	22
Sai lầm 2: Component quá lớn	22
Sai lầm 3: Cyclic dependencies tolerated	22
Sai lầm 4: Component theo layer (horizontal slicing)	22
Tóm tắt	22
Tổng kết Phần 4	23

4.1 Tổng quan Phần 4: Quality Attributes

Tóm tắt một dòng: Functional requirement nói hệ thống phải làm gì, còn Quality Attribute nói hệ thống phải làm việc đó tốt đến mức nào. Với production ML, accuracy chỉ là một phần nhỏ. Bạn còn phải quan tâm latency, freshness, reliability, reproducibility, observability, privacy và cost.

Câu hỏi mở đầu

Hãy tưởng tượng bạn train được một churn prediction model có AUC rất cao. Trong notebook, mọi thứ trông tuyệt vời: data clean, validation split đúng, metric đẹp, biểu đồ rõ ràng. Bạn demo cho business, mọi người hài lòng. Sau đó team đưa model vào production.

Một tuần sau, hệ thống bắt đầu có vấn đề:

- Prediction trả về quá chậm, CRM page bị lag.
- Feature `last_7d_purchase_count` đôi khi trễ 24 giờ.
- Data schema thay đổi, pipeline vẫn chạy nhưng feature bị null nhiều hơn trước.
- Model mới tốt trên validation nhưng tệ hơn trong production, rollback mất cả ngày.
- Không ai biết prediction của user A được tạo bởi model version nào.
- Log có đủ thông tin để debug, nhưng lại chứa dữ liệu cá nhân.

Điều đáng chú ý là model không hề “sai” theo nghĩa Data Science truyền thống. Accuracy vẫn có thể tốt. Nhưng **hệ thống dùng model đó không đạt quality attributes cần thiết**.

Đây là lý do Phần 4 quan trọng. Software Architecture không chỉ hỏi “hệ thống có chức năng predict không?”. Nó hỏi: predict có nhanh không, ổn định không, quan sát được không, giải thích được không, rollback được không, chi phí có chịu được không, và có an toàn dữ liệu không.

Quality Attribute là gì?

Một **Quality Attribute** (QA), còn gọi là **Architecture Characteristic** hoặc **Non-functional Requirement**, là một thuộc tính mô tả *cách* hệ thống phải hoạt động, được xây dựng hoặc được vận hành.

So sánh đơn giản:

Loại requirement	Câu hỏi	Ví dụ web app	Ví dụ ML/data system
Functional Requirement	Hệ làm gì?	User đặt được order	Hệ predict churn score
Quality Attribute	Hệ làm tốt đến mức nào?	p95 checkout < 500ms	p95 inference < 100ms, feature freshness < 1h

Một sai lầm rất phổ biến là coi QA như phần phụ, kiểu “xong feature rồi tối ưu sau”. Trong kiến trúc, QA thường quyết định hình dạng hệ thống ngay từ đầu. Nếu bạn cần batch score mỗi đêm, kiến trúc khác. Nếu bạn cần online fraud detection dưới 100ms, kiến trúc khác hoàn toàn. Nếu bạn cần audit từng prediction vì có rủi ro pháp lý, kiến trúc lại khác nữa.

Nếu bạn đến từ Data Science

Trong Data Science, bạn đã quen với metric. Vì vậy hãy nghĩ Quality Attribute như **metric của hệ thống production**, không phải metric của model offline.

Model metric quen thuộc	System metric tương ứng
Accuracy, F1, AUC	Business correctness và model quality
Training time	Pipeline duration, resource cost
Data split reproducibility	End-to-end reproducibility
Feature importance	Explainability, auditability
Validation score	Online performance monitoring
Dataset freshness	Data freshness SLA
Experiment tracking	Traceability, lineage, rollback

Điểm khác biệt là system metric thường liên quan tới nhiều component, không nằm trong một file training. Ví dụ p95 inference latency phụ thuộc vào API gateway, feature lookup, model runtime, CPU/GPU allocation, network, serialization và logging. Vì vậy QA là cầu nối giữa Data Science và Software Architecture.

Phần 4 sẽ dạy gì?

Bài 4.2: Functional vs Non-functional Requirements

Bài này giúp bạn phân biệt “hệ thống phải làm gì” với “hệ thống phải làm như thế nào”. Với ML system, functional requirement có thể là “trả về churn score cho mỗi customer”. Non-functional requirement mới là phần làm kiến trúc khó: score phải cập nhật mỗi ngày hay mỗi giây, response time bao nhiêu, có cần explainability không, có cần lưu lại prediction để audit không.

Bài 4.3: Identifying Architecture Characteristics

Bài này dạy cách tìm QA từ requirement document, stakeholder interview và business context. Với DS, stakeholder không phải lúc nào cũng nói “tôi cần feature freshness dưới 15 phút”. Họ có thể nói “đừng dùng thông tin cũ để chặn giao dịch gian lận”. Nhiệm vụ của architect là dịch câu đó thành QA đo được.

Bài 4.4: Component-Based Thinking

Sau khi biết QA cần tối ưu, bạn phải map chúng xuống component. Ví dụ nếu freshness quan trọng, feature pipeline và feature store trở thành component trung tâm. Nếu rollback quan trọng, model registry và deployment strategy phải được thiết kế rõ. Nếu privacy quan trọng, logging và data retention không thể để tùy hứng.

Bảng QA phổ biến

Operational QA: xảy ra khi hệ thống chạy

QA	Nghĩa ngắn	Ví dụ ML/data system
Performance	Response time, throughput	p95 inference < 100ms, score 1M records trong 30 phút
Scalability	Load tăng vẫn chạy tốt	Tăng từ 1k lên 100k prediction/phút
Availability	Hệ sẵn sàng phục vụ	Serving endpoint uptime 99.9%
Reliability	Kết quả đúng, không corrupt	Không score thiếu user, không dùng feature sai schema
Recoverability	Phục hồi sau lỗi	Rollback model trong 10 phút
Observability	Nhìn được state hệ thống	Dashboard latency, drift, error rate, feature null rate
Security	Bảo vệ dữ liệu và quyền truy cập	Chỉ service được phép mới đọc feature chứa PII

Structural QA: ảnh hưởng cách code và team phát triển

QA	Nghĩa ngắn	Ví dụ ML/data system
Maintainability	Dễ sửa, dễ thêm feature	Thêm model mới không phải sửa toàn pipeline
Modularity	Boundary rõ	Tách ingestion, feature, training, serving, monitoring
Testability	Dễ test tự động	Test data validation, feature transform, inference contract
Deployability	Dễ release	Deploy model version mới không downtime
Reusability	Dùng lại được	Feature transform dùng chung training và serving
Evolvability	Dễ thay đổi theo thời gian	Thay model framework từ sklearn sang PyTorch ít ảnh hưởng

Cross-cutting QA: cắt ngang nhiều phần

QA	Nghĩa ngắn	Ví dụ ML/data system
Cost	Tiền cloud, GPU, storage, con người	GPU serving có đáng so với CPU không
Privacy	Bảo vệ dữ liệu cá nhân	Mask PII trong logs và training data
Auditability	Trace ai làm gì, khi nào	Prediction được tạo bởi model/data version nào
Reproducibility	Chạy lại ra cùng kết quả	Rebuild model từ data snapshot và code version
Data lineage	Biết data đi từ đâu tới đâu	Feature X đến từ bảng nào, job nào, version nào

QA	Nghĩa ngắn	Ví dụ ML/data system
Freshness	Data mới tới mức nào	Feature trong online store không cũ quá 5 phút
Explainability	Giải thích được quyết định	Lý do transaction bị flag fraud

Vì sao không thể chọn tất cả?

Nếu chỉ nhìn danh sách trên, ta dễ muốn tất cả: latency thấp, cost thấp, freshness cao, explainability cao, privacy tốt, scale vô hạn, deploy dễ. Nhưng architecture là trade-off. Tối ưu một thứ thường làm thứ khác tệ đi.

Accuracy vs Latency

Một deep model lớn có thể accuracy tốt hơn, nhưng inference chậm và cần GPU. Một logistic regression có thể accuracy thấp hơn một chút nhưng chạy nhanh, explainable và rẻ. Nếu use case là fraud detection real-time, latency có thể quan trọng hơn 0.5% AUC.

Freshness vs Cost

Update feature mỗi phút giúp model phản ứng nhanh hơn. Nhưng compute, orchestration và monitoring đắt hơn. Nếu churn prediction chỉ dùng cho campaign mỗi sáng, feature freshness 24 giờ có thể đủ. Nếu fraud detection, 24 giờ là không chấp nhận được.

Privacy vs Observability

Log đầy đủ request, feature vector và prediction giúp debug rất tốt. Nhưng nếu log chứa PII, bạn tạo rủi ro security và compliance. Architecture phải quyết định log cái gì, mask cái gì, retention bao lâu, ai được xem.

Simplicity vs Scalability

Một cron job batch scoring có thể đơn giản và dễ maintain. Kafka streaming pipeline scale tốt hơn và realtime hơn, nhưng vận hành khó hơn nhiều. Nếu business không cần realtime, chọn streaming chỉ vì nghe hiện đại là over-engineering.

Explainability vs Performance

Một model explainable giúp stakeholder tin tưởng và compliance dễ hơn. Nhưng model đó có thể kém performance hơn. Ngược lại, model phức tạp có thể tốt hơn về metric nhưng khó giải thích. Câu trả lời phụ thuộc domain: recommendation có thể chấp nhận ít explainability hơn, credit scoring thì không.

Cách operationalize QA

Một QA chưa đo được thì chưa đủ dùng để ra quyết định. “Hệ phải nhanh” là mơ hồ. “p95 inference latency dưới 100ms ở 500 requests/second” mới đủ rõ.

QA mơ hồ	Viết lại đo được
Model phải nhanh	p95 inference latency < 100ms, p99 < 250ms

QA mơ hồ	Viết lại đo được
Data phải mới	Feature freshness < 15 phút cho 99% records
Pipeline phải ổn	Daily training pipeline success rate > 99%
Dễ rollback	Rollback model version trong < 10 phút
Dễ debug	100% predictions trace được model version + feature snapshot
Chi phí hợp lý	Serving cost < \$0.10 / 1000 predictions
Không leak dữ liệu	0 PII trong application logs, scan hằng ngày

Operationalize là bước biến cuộc tranh luận cảm tính thành engineering. Nếu stakeholder nói “cần realtime”, hãy hỏi realtime nghĩa là 50ms, 5 giây, 5 phút hay 1 giờ. Bốn con số này dẫn tới bốn kiến trúc rất khác nhau.

Ví dụ: chọn architecture cho churn prediction

Giả sử business cần churn score để đội CRM gọi khách hàng có nguy cơ rời bỏ.

Option 1: Batch inference mỗi đêm



Hình 1: Sơ đồ 9

QA đạt tốt:

- Cost thấp.
- Đơn giản.
- Reproducibility tốt.
- Dễ audit.

QA hy sinh:

- Freshness không cao.
- Không phù hợp nếu cần phản ứng theo hành vi realtime.

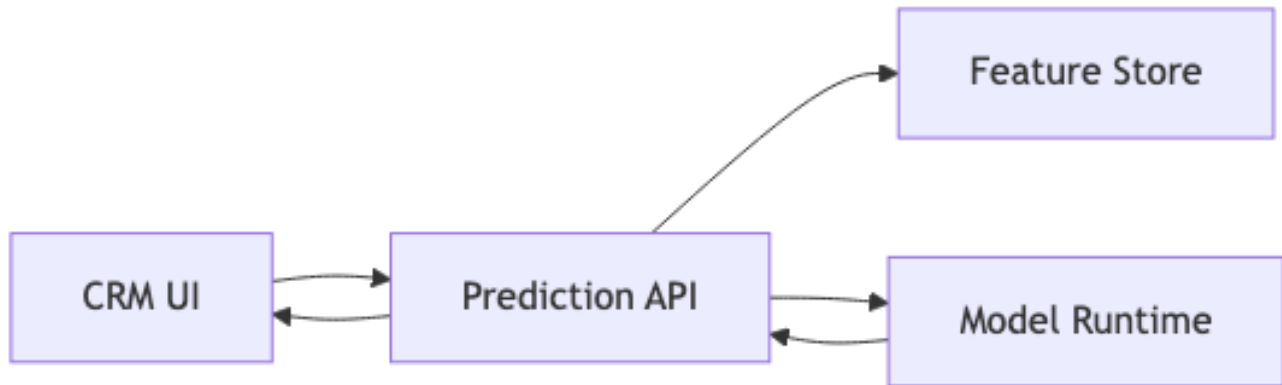
Option 2: Online inference khi CRM mở profile

QA đạt tốt:

- Prediction fresh hơn.
- Có thể personalize theo request hiện tại.
- Dễ tích hợp vào nhiều channel.

QA hy sinh:

- Latency cần quản lý.
- Availability của API trở nên quan trọng.
- Cost và operational complexity cao hơn.



Hình 2: Sơ đồ 10

Không có option nào “đúng tuyệt đối”. Nếu CRM gọi khách mỗi sáng theo batch list, Option 1 đủ tốt. Nếu app cần hiển thị offer realtime khi user đang tương tác, Option 2 hợp lý hơn. Quyết định phụ thuộc QA ưu tiên.

Architect's responsibility

Một quyết định kiến trúc tốt nên làm ba việc:

1. **Nói rõ QA ưu tiên.** Ví dụ freshness < 15 phút quan trọng hơn cost.
2. **Nói rõ trade-off.** Ví dụ dùng streaming tăng freshness nhưng tăng operational cost.
3. **Ghi lại quyết định.** Ví dụ viết ADR: “Chọn batch inference cho churn scoring vì business action theo daily campaign”.

Nếu architect ra quyết định mà không nói QA, quyết định đó rất dễ thành tranh luận sở thích: người thích Kafka, người thích cron, người thích microservices, người thích monolith. QA giúp team quay về câu hỏi đúng: hệ thống này cần tối ưu điều gì?

Self-check

Trước khi sang bài tiếp theo, hãy tự trả lời:

1. Với một model bạn từng làm, 5 QA production quan trọng nhất là gì?
2. Metric offline của model có mâu thuẫn với system metric nào không?
3. Nếu business nói “cần realtime”, bạn sẽ hỏi lại những câu nào để operationalize?
4. Nếu phải giảm cloud cost 50%, QA nào có thể bị ảnh hưởng?
5. Bạn có trace được prediction từ model version, data version và code version không?

Cách đọc phần này

Đọc tuần tự 4.2 □ 4.3 □ 4.4. Nếu bạn đến từ Data Science, hãy đọc Phần 4 trước khi đi quá sâu vào microservices. Nhiều quyết định distributed chỉ có ý nghĩa sau khi bạn biết rõ QA cần tối ưu.

Mục tiêu thực hành sau phần:

- List 5-7 QA ưu tiên cho một hệ bạn biết.
- Viết mỗi QA thành metric đo được.
- Vẽ component diagram thể hiện QA đó nằm ở component nào.
- Viết một ADR ngắn cho quyết định batch vs online, hoặc REST vs Kafka.

Bài tiếp: [Functional vs Non-functional Requirements](#).

4.2 Functional vs Non-functional Requirements

Tóm tắt một dòng: Functional requirement nói “hệ phải làm X”; Quality attribute nói “hệ phải làm X tốt cỡ nào”. Architecture được quyết định bởi QA chứ không phải bởi functional requirement, vì cùng functionality có thể được implement bằng vô số architecture khác nhau.

Nếu bạn đến từ Data Science

Trong ML system, functional requirement thường nghe rất đơn giản: “hệ thống trả về churn score”, “hệ thống phát hiện fraud”, “hệ thống recommend sản phẩm”. Nhưng phần làm kiến trúc khó nằm ở NFR: score phải trả trong bao lâu, feature được phép cũ bao lâu, prediction có cần explain không, có cần audit model version không, pipeline fail thì recover thế nào.

Vì vậy đừng dừng ở câu “model predict được”. Hãy hỏi tiếp: predict cho bao nhiêu request mỗi giây, với p95 latency bao nhiêu, khi schema đổi thì phát hiện ở đâu, khi model drift thì ai biết, rollback mất bao lâu. Những câu hỏi này biến một model thành production system.

Câu hỏi mở đầu

Hai team build cùng một e-commerce site. Functional requirement giống hệt: catalog, cart, checkout, payment.

Team A architect:

- Monolithic Rails app.
- PostgreSQL.
- Deploy 1 server EC2 t3.medium.
- Cost: \$30/tháng.

Team B architect:

- 12 microservices (Catalog, Cart, Inventory, Payment, ...) trên Kubernetes.
- PostgreSQL + Redis + Elasticsearch + Kafka.
- Multi-region active-active deployment.
- Cost: \$15,000/tháng.

Cả hai đều “đáp ứng functional requirement”, user có thể browse, add to cart, checkout. Nhưng architecture khác nhau 500 lần về cost. Tại sao?

Vì *non-functional requirement* khác nhau:

- Team A: ~100 user, không cần multi-region, downtime 1 giờ/tháng OK.
- Team B: 10M user, 99.99% uptime, p99 latency < 200ms, scale to 10x traffic in 5 phút.

Architecture decision được driven bởi NFR, không phải bởi functional requirement. Đây là một trong những insight quan trọng nhất của Software Architecture.

Định nghĩa formal

Functional Requirement (FR)

“What the system must do.”, Hành vi observable mà hệ phải có.

Ví dụ: - “User có thể đăng ký account bằng email + password.” - “Admin có thể export CSV danh sách user.” - “Hệ tự động gửi email khi order được ship.”

FR pass/fail qua acceptance test: hành vi xảy ra hoặc không.

Non-functional Requirement (NFR)

“How well the system must do it.”, Constraint trên cách hệ behave.

Ví dụ: - “API response p99 < 200ms cho 95% endpoints.” - “Uptime ≥ 99.9% (3 phút downtime/tháng).” - “Hỗ trợ 10,000 concurrent users.” - “Code 80%+ unit test coverage.”

NFR đo qua benchmark, monitoring, code metric. Có thể “đạt mức nào đó”, không pass/fail binary.

Architecture Characteristic (Mark Richards)

Mark Richards trong *Fundamentals of Software Architecture* dùng từ “architecture characteristic” thay vì NFR. Định nghĩa:

“An architecture characteristic meets three criteria:”

1. Specifies a **non-domain design consideration**.
2. **Influences some structural aspect** of the design.
3. **Critical or important** to application success.

Phân tích 3 tiêu chí:

1. Non-domain: không thuộc về business logic. Vd “Order phải có tax” là domain rule. “Order processing latency < 200ms” là non-domain, không nói về domain order, nói về *cách* processing.

2. Structural impact: yêu cầu thay đổi structure của hệ. Vd “100 concurrent user” có thể serve bởi single server. “10M concurrent user” require horizontal scaling, load balancer, có lẽ CDN, structure khác hoàn toàn.

3. Critical: nếu không đạt, hệ fail. Vd: “Pinterest cần search latency < 100ms”, nếu 5 giây thì user rời ngay. “Internal admin tool 2-second latency OK”, không critical.

Mọi NFR architectural đều thoả 3 tiêu chí. Một số NFR không thoả tiêu chí 2 (vd “code 80% test coverage” không structural) □ không architectural.

Implicit vs Explicit Characteristics

Explicit

Stakeholder nói rõ trong requirement doc. Vd: “API p99 < 200ms”, viết trong SLA.

Implicit

Stakeholder không nói nhưng *vẫn expect*. Vd:

- User expect “hệ không bị hack”, security implicit (trừ khi product là hack tool, ha ha).
- User expect “data không bị mất khi crash”, reliability implicit.
- Developer expect “build < 5 phút”, maintainability implicit.

Architect job: identify implicit characteristics. Stakeholder không có ngôn ngữ để articulate chúng, architect đào ra.

Pattern: với mỗi feature, hỏi:

- “Nếu feature này down 1 ngày, user phản ứng sao?”
- “Nếu data sai 1%, ảnh hưởng gì?”
- “Nếu attacker exploit endpoint này, hậu quả?”

- “Nếu hệ scale lên 10x, gì breaks first?”

Câu trả lời tiết lộ implicit characteristics.

“Architecture is the X-ilities”

Trong industry, NFR thường gọi là “-ilities” vì nhiều cái kết thúc bằng -ility:

- **Reliability**
- **Scalability**
- **Maintainability**
- **Testability**
- **Deployability**
- **Observability**
- **Auditability**
- **Availability**

Quote nổi tiếng (không rõ tác giả): “Architecture is the X-ilities. Get them right and your system works. Get them wrong and your system collapses.”

Reason: -ilities là cái khó retro-fit. Function add thêm vào hệ cũ được. Performance/scalability/security bake vào ngay từ đầu mới đảm bảo.

Ví dụ thực hành: từ FR □ NFR □ Architecture

Use case: Hệ booking phim cinema

Functional requirements: - User search phim theo ngày, rạp, thể loại. - User chọn ghế, payment. - User nhận email confirmation + QR code. - Admin manage suất chiếu, giá vé.

Non-functional requirements (cần đào):

- *Performance*: search response < 500ms (user wait không bị nản).
- *Concurrency*: thứ bảy 7pm peak 50,000 concurrent user nationwide.
- *Consistency*: 2 user không được book cùng ghế (strong consistency for seat selection).
- *Availability*: ≥ 99.5% uptime, đặc biệt cao ngày cuối tuần.
- *Scalability*: peak khoảng 10x normal, scale up trong < 5 phút.
- *Security*: payment information theo PCI DSS.
- *Auditability*: log mọi transaction cho refund support.

Architecture implications:

- *Concurrency 50k + strong consistency seat*: cần seat reservation pattern (Redis SETNX hoặc database row lock). Không cache seat availability.
- *Peak 10x*: auto-scaling group, scale-out architecture (stateless services).
- *PCI DSS*: payment processing isolated service, không touch app-level code. Stripe/local gateway.
- *Audit*: append-only event log (Event Sourcing optional, audit log mandatory).

Quyết định kiến trúc *suy ra* từ NFR. Khác NFR □ khác architecture.

NFR khó hơn FR

NFR thường:

Khó measure

“Maintainable” cụ thể là gì? Cyclomatic complexity? Lines of code? Time to onboard new dev?

Solution: operationalize NFR bằng metric cụ thể. “Maintainable” = “new dev onboard < 2 tuần đến first PR” + “average bugfix time < 4 giờ” + “cyclomatic complexity < 15 per method”.

Khó test

Performance test cần load generator. Scalability cần production-like environment. Security cần penetration test. Cost cao.

Khó negotiate với business

Business muốn “everything fast and cheap”. Architect job: present trade-off, force prioritization.

Tool: ATAM, CQA (Continuous Quality Assurance) workshop, sẽ học ở 4.3.

Conflict nhau

Đã nói ở 4.1: cặp trade-off. Không pick all.

Sai lầm thường gặp

Sai lầm 1: Bỏ qua NFR trong design phase

Team build feature theo functional spec, không hỏi NFR. Production ship rồi mới đau: too slow, không scale, security holes.

Fix: NFR phải elicited *trước* khi viết code. Workshop 1-2 giờ với business để extract NFR.

Sai lầm 2: NFR không measurable

“Hệ phải fast.” □ useless. Phải định lượng: “p99 < 200ms”.

Sai lầm 3: Treat tất cả NFR như nhau

Stakeholder say “tất cả NFR đều critical”. Sai. Phải force priority. 5-7 NFR top.

Sai lầm 4: NFR không liên kết với business value

NFR phải justify bằng business impact. “p99 < 200ms vì study cho thấy mỗi 100ms tăng latency làm conversion giảm 1%”. Không justify được □ cắt khỏi requirement.

Tóm tắt

- **FR**: hệ làm gì. **NFR**: hệ làm tốt cỡ nào.
- **Architecture decision driven bởi NFR**, không phải FR.
- **Architecture characteristic** (Richards): non-domain + structural impact + critical.
- **Implicit characteristics**: architect đào ra từ stakeholder.
- **NFR khó**: measure, test, negotiate, conflict.

Bài tiếp: [Identifying Architecture Characteristics](#), kỹ thuật extract NFR từ requirement.

4.3 Xác định Architecture Characteristics

Tóm tắt một dòng: Đừng pick “all the -ilities”. Chọn 3-7 QA top driven bởi business priority, document chúng có thể đo được, dùng workshop với stakeholder để negotiate khi xung đột.

Nếu bạn đến từ Data Science

Stakeholder hiếm khi nói thẳng bằng thuật ngữ architecture. Họ sẽ không nói “tôi cần feature freshness SLA 5 phút”. Họ có thể nói “đừng để hệ thống chặn giao dịch dựa trên thông tin cũ”. Nhiệm vụ của bạn là dịch câu đó thành Quality Attribute đo được.

Một số câu hỏi hữu ích: prediction stale bao lâu thì gây hại? Nếu model sai, ai chịu rủi ro? Có cần giải thích từng prediction không? Data chứa PII không? Label về trễ bao lâu? Nếu pipeline training fail, business có chấp nhận dùng model cũ thêm một ngày không? Từ câu trả lời, bạn extract được freshness, explainability, privacy, recoverability và reliability.

Vì sao “không quá 7”?

Cognitive limit. Người ta nhớ và optimize được khoảng 5-9 thứ cùng lúc (Miller’s 7 ± 2 rule). Architect đặt 15 QA “quan trọng” □ 15 QA đều không thực sự được tối ưu.

Quy tắc: **Top 5-7 QA**. Còn lại là implicit (default expectations như “không crash”, “không leak password”) nhưng không driven decision.

Tỷ lệ thực hành:

- 3 QA: lý tưởng cho startup MVP. Vd: “time-to-market, simplicity, cost”.
- 5 QA: ổn cho hệ trung. Vd: “performance, scalability, availability, maintainability, security”.
- 7 QA: max. Vd: thêm “auditability, observability”.
- 10+ QA: bạn đang nói dối rằng mọi thứ đều ưu tiên. Force prioritize.

Pipeline identify

Bước 1: Đọc requirement document

Highlight mọi từ tiềm năng là NFR. Vài keyword:

- “fast”, “responsive”, “real-time” □ performance
- “scale”, “grow”, “millions” □ scalability
- “uptime”, “always available”, “24/7” □ availability
- “secure”, “encrypted”, “compliant” □ security
- “audit”, “trace”, “log” □ auditability
- “easy to deploy”, “frequent release” □ deployability
- “low cost”, “budget” □ cost

Lưu ý: business owner thường không dùng từ kỹ thuật. “Hệ phải always work” có thể nghĩa availability hoặc reliability. Phải hỏi để clarify.

Bước 2: Stakeholder interview

Schedule meeting 1-2h với:

- Product Owner / PM (business priorities).
- Engineering Manager (team capability).
- Customer Success (user pain points hiện tại).
- Optional: end user (vd: focus group cho consumer product).

Câu hỏi mẫu:

- “Trong 3 năm tới, hệ này có khả năng phát triển ra sao? 10x user? 100x?” □ scalability priority.
- “Nếu hệ down 1 giờ giữa giờ làm, ai bị ảnh hưởng và thiệt hại bao nhiêu?” □ availability priority.
- “Tốc độ release feature hiện tại là tuần? tháng? Có cần nhanh hơn không?” □ time-to-market + deployability.
- “Có khả năng audit / compliance requirement (GDPR, SOC2, PCI)?” □ compliance/auditability.
- “Hệ này lưu data nhạy cảm? Loại nào?” □ security/privacy.

Bước 3: Implicit characteristics

Như đã nói ở Bài 4.2, ngoài explicit, còn implicit. Default implicit cho most systems:

- Reliability: không data corruption.
- Security: basic auth, no XSS/SQLi.
- Maintainability: ai cũng có thể onboard.
- Observability: logs + basic metrics.

Implicit không cần “top 7”, nhưng cần *baseline*. Architect đảm bảo baseline được respected.

Bước 4: Priority với business

Workshop format:

1. List 10-15 candidate QA (mix explicit + implicit).
2. Business stakeholder rank: must-have / should-have / nice-to-have.
3. Push back: nếu mọi cái “must-have”, force chọn top 5.
4. Output: top 5-7 QA + acceptable range cho mỗi cái.

Ví dụ output:

QA	Priority	Target
Performance	Must	p99 < 200ms cho 95% endpoint
Availability	Must	99.9% uptime
Security	Must	Compliance OWASP Top 10
Scalability	Should	Scale 10x trong 1 năm
Maintainability	Should	Onboard new dev < 2 tuần
Cost	Should	< \$5k/tháng infra cho 50k users

Document này là “north star” cho mọi architecture decision.

Bước 5: Operationalize

Translate QA thành metric đo được.

QA	Cách đo
Performance	p50, p95, p99 latency từ APM (Datadog, New Relic)
Availability	Uptime % từ monitoring (Pingdom, internal)
Scalability	Max concurrent users từ load test

QA	Cách đo
Maintainability	Mean time to PR-merged, cyclomatic complexity
Deployability	Time from commit to production
Security	Pen-test results, dependency vulnerability count
Cost	Monthly cloud bill

Mỗi QA nên có:

- Metric đo (number).
- Cách đo (tool).
- Threshold pass/fail (vd: p99 < 200ms = pass; > 200ms = alert).
- Frequency review (vd: weekly architecture review).

Không operationalize được □ QA không “real”, chỉ là khẩu hiệu.

Hierarchy của Characteristics

Có thể group QA theo 3 layer:

Operational (Runtime characteristics)

QA về *cách hệ chạy* trong production:

- Performance
- Scalability
- Availability
- Reliability
- Security
- Recoverability

Đo bằng monitoring tool.

Structural (Development characteristics)

QA về *cách hệ được build*:

- Maintainability
- Testability
- Deployability
- Modularity
- Reusability

Đo bằng code metric + dev workflow metric.

Cross-cutting

QA xuyên qua mọi layer:

- Cost
- Time-to-market
- Auditability
- Observability

- Compliance (GDPR, HIPAA, ...)
- Accessibility
- i18n

Đo theo context.

Insight: thường top 5-7 QA distributed across 3 layer. Vd:

- 2 operational (performance, availability).
- 2 structural (maintainability, deployability).
- 1-2 cross-cutting (cost, compliance).

Distribution này khoẻ. Tất cả ở 1 layer là warning (bias).

CQA Worksheet (Quality Attribute Workshop simplified)

Format compact để run workshop 60-90 phút:

QA	Scenario (concrete)	Source	Response	Measure
Performance	User search product on mobile during peak	User	Result returned	p99 < 300ms
Scalability	Black Friday traffic 10x normal	System	All requests served	< 5% error rate
Availability	DB primary fails	System	Failover to replica	< 30s outage
Maintainability	Junior dev fix critical bug	Dev	PR merged	< 4 hours
Security	Attacker tries SQL injection	Attacker	Request blocked	0 successful attacks/year

Mỗi row là một “QA scenario”. Scenario cụ thể để test hơn QA abstract. Output workshop = bảng scenarios với measure rõ ràng.

Reference: ATAM (Architecture Tradeoff Analysis Method) của SEI Carnegie Mellon, full version dùng cho hệ critical. Quality Attribute Workshop simplified dùng cho hệ thường.

Sai lầm thường gặp

Sai lầm 1: “Mọi thứ đều quan trọng”

Phổ biến nhất. Stakeholder nói tất cả must-have. Architect không push back → end up với 15 QA = không QA nào được tối ưu thật.

Fix: force ranking. “Nếu chỉ chọn 3, bạn chọn cái nào?”.

Sai lầm 2: QA không measurable

“Hệ phải user-friendly.” → không đo được. Refine thành: “task completion rate > 85% trong usability test với 10 user.”

Sai lầm 3: Implicit không document

Implicit characteristics được architect assume nhưng không communicate. Sau 1 năm team thay đổi, no one biết. Document chúng vào “implicit baseline” section của architecture doc.

Sai lầm 4: QA conflict không resolve

List “performance + security + simplicity” mà không acknowledge chúng có thể conflict. Khi conflict xảy ra trong development, không có guidance để pick.

Fix: trong workshop, present cặp QA có thể conflict và force prioritization. Vd: “Khi performance vs security xung đột, mình chọn cái nào?”.

Sai lầm 5: Không revisit

QA được set 1 lần ở project kickoff rồi quên. Business priority có thể thay đổi (vd: COVID làm e-commerce ưu tiên availability hơn cost).

Fix: revisit QA mỗi quarter hoặc khi có major business change.

Tóm tắt

- **Top 5-7 QA**, không nhiều hơn.
- **Pipeline**: read requirement → interview → identify implicit → priority workshop → operationalize.
- **Hierarchy**: operational + structural + cross-cutting.
- **CQA worksheet**: scenario-based, measurable.
- **Sai lầm**: “mọi thứ quan trọng”, QA không measurable, implicit không doc, conflict không resolve, không revisit.

Bài tiếp (cuối Phần 4): [Component-Based Thinking](#), map QA xuống component.

4.4 Component-Based Thinking

Tóm tắt một dòng: Component là building block của architecture (lớn hơn class, có boundary rõ, có thể deploy/test độc lập). Identify component bằng entity-based hoặc workflow-based; tổ chức theo 3 cohesion + 3 coupling principles của Robert C. Martin.

Nếu bạn đến từ Data Science

Component trong ML/data system thường là ingestion, validation, feature store, trainer, evaluator, model registry, serving API, monitoring. Mỗi component nên có trách nhiệm rõ và owner rõ. Nếu feature transformation vừa nằm trong training notebook vừa copy sang serving code, bạn đã tạo coupling nguy hiểm giữa training và serving.

Component thinking giúp bạn hỏi đúng: feature definition sống ở đâu? Ai own schema? Model registry lưu những metadata nào? Serving API đọc feature online hay tự tính? Monitoring nhận prediction log từ đâu? Những câu hỏi này quyết định architecture nhiều hơn việc chọn model algorithm.

Component là gì?

Robert C. Martin định nghĩa trong *Clean Architecture*:

“Components are the units of deployment. They are the smallest entities that can be deployed as part of a system.”

Examples cụ thể:

- Trong Java: JAR file.
- Trong .NET: DLL file.
- Trong Ruby: Gem.
- Trong JavaScript: npm package.
- Trong Python: wheel / module.
- Trong microservices: deployed service.

Key property: **independently deployable**. Component có boundary rõ, vào ra qua interface công khai. Bên trong là chi tiết.

Component khác:

- **Class**: nhỏ hơn. Một component thường chứa nhiều class.
- **Module** (theo nghĩa code organization): tương đương component ở một số ngữ cảnh, nhưng “module” thường mức code, “component” thường mức deployment.

Component vs Module, phân biệt

Nuance:

- **Module**: package code organization. Mục đích: navigation + namespace. Vd: Python package `myapp.orders`.
- **Component**: deployable unit. Mục đích: deployment + versioning. Vd: `orders-service.jar`.

Trong monolith, có nhiều module nhưng 1 component (1 deployable). Trong microservices, mỗi service là 1 component.

Phần 5 (Architecture Styles) sẽ trở lại: mỗi style có cách khác nhau để compose component.

Identify Component

Hai phương pháp phổ biến:

1. Entity-based (theo domain entity)

Mỗi entity quan trọng □ component own entity đó.

Vd e-commerce:

- CustomerService component own Customer entity.
- OrderService component own Order entity.
- ProductService component own Product entity.
- PaymentService component own Payment entity.

Lợi ích: bounded context rõ. Mỗi component có data ownership.

Hạn chế: nếu một workflow dùng nhiều entity (vd: “place order” dùng Customer + Order + Product + Inventory + Payment), workflow phải cross 5 component □ coupling cao.

2. Workflow-based (theo use case)

Mỗi workflow quan trọng □ component own workflow đó.

Vd:

- OrderPlacementService cover toàn bộ flow “place order” (orchestrate Customer + Inventory + Payment internally).
- CustomerOnboardingService cover flow “đăng ký + verify email + setup profile”.

Lợi ích: workflow stay coherent, ít cross-component call.

Hạn chế: data ownership phức tạp (Customer entity bị edit bởi cả OrderService và OnboardingService).

Combined approach

Thực tế thường mix:

- **Aggregate entities** thành Bounded Context (DDD): OrderManagement context own Order, OrderItem, OrderStatus entities.
- **Workflow** dùng cross-context: CheckoutFlow orchestrate OrderManagement + Payment + Shipping.

Mỗi approach có pros/cons. Quyết định theo: số workflow cross-entity, frequency thay đổi, team structure.

Component Cohesion Principles (Robert C. Martin)

Ba nguyên tắc cho việc *cái gì nên ở cùng component*:

REP, Reuse/Release Equivalence Principle

“The granule of reuse is the granule of release.”

Diễn đạt: bất cứ thứ gì bạn release together (cùng version) phải reusable together. Người dùng component không thể chọn release một phần của component, họ get tất cả hoặc không gì.

Hệ quả: classes trong component nên có *common purpose*. Vd: react-router package chỉ chứa routing-related code, không lẫn react-i18n.

CCP, Common Closure Principle

“Gather into components those classes that change for the same reasons and at the same times.”

Diễn đạt: class thay đổi cùng nhau nên ở cùng component. Khi requirement thay đổi, ideally chỉ 1 component bị touch.

Hệ quả: chia component theo *axis of change*. Vd: nếu UI hay thay đổi separately từ business logic, tách thành 2 component.

Note: CCP scale up SRP từ class lên component.

CRP, Common Reuse Principle

“Don’t force users of a component to depend on things they don’t need.”

Diễn đạt: nếu user dùng class X từ component C, user shouldn’t be forced to depend trên class Y trong C mà user không dùng.

Hệ quả: classes có usage pattern khác nhau nên ở component khác nhau. Vd: Database và Cache thường được dùng cùng nhau □ cùng component OK. Nhưng EmailSender không được dùng cùng □ tách component.

Note: CRP scale up ISP lên component.

Tension giữa 3 principles

Trade-off:

- REP + CCP push toward larger components (more in 1 component).
- CRP push toward smaller components (less in 1 component).

Triangle:

3 cạnh không thể đều đạt 100%, phải balance. Nguyên tắc:

- Early stage: prioritize CCP + REP (tooling/team focus on flexibility).
- Mature stage: shift to CRP (tooling/team focus on stability).

Component Coupling Principles

Ba nguyên tắc cho *cách component depend lẫn nhau*:

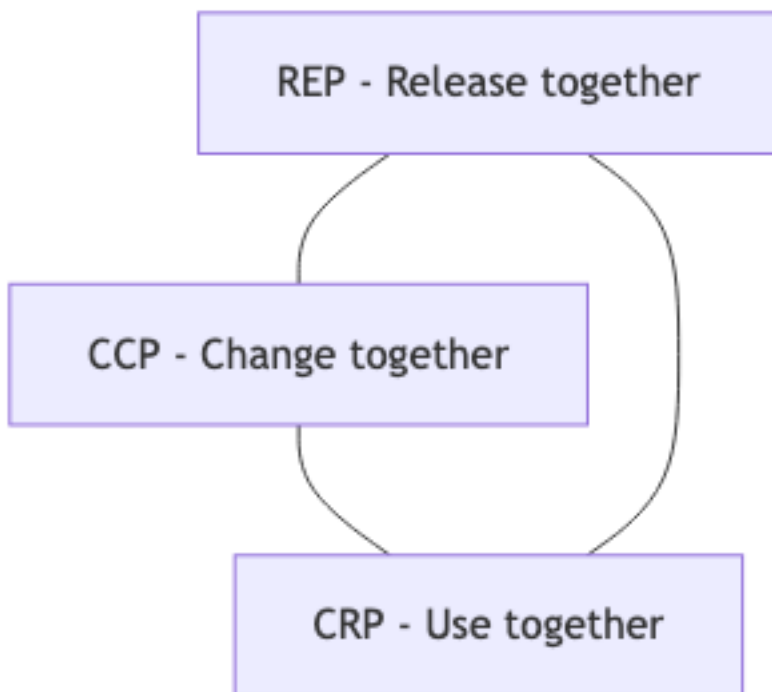
ADP, Acyclic Dependencies Principle

“Allow no cycles in the component dependency graph.”

Nếu $A \square B \square C \square A \square$ cycle □ cả 3 phải release cùng nhau, mất ý nghĩa tách component.

Detect: tool như jdepend (Java), madge (JS), pydeps (Python) visualize dependency graph + báo cycles.

Fix: - **Move dependency**: tách class gây cycle ra component khác. - **Dependency Inversion**: introduce abstraction để break cycle (DIP từ Bài 2.7).



Hình 3: Sơ đồ 11

SDP, Stable Dependencies Principle

“Depend in the direction of stability.”

Component thay đổi nhiều (volatile) nên depend vào component thay đổi ít (stable). Không ngược lại.

Lý do: nếu stable component depend volatile, mỗi lần volatile thay đổi □ stable phải re-release □ stable hết stable.

Đo stability: ratio $I = C_e / (C_a + C_e)$ với:

- C_a (afferent coupling): số component depend vào component này.
- C_e (efferent coupling): số component component này depend vào.
- $I = 0$: maximally stable (chỉ bị depend, không depend ai).
- $I = 1$: maximally unstable.

SAP, Stable Abstractions Principle

“A component should be as abstract as it is stable.”

Component stable nên abstract (interface), component volatile có thể concrete.

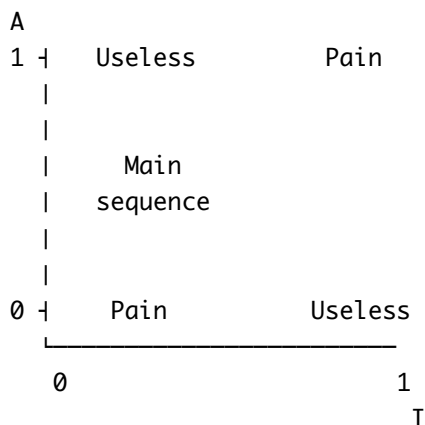
Lý do: stable component không nên chứa code chi tiết (vì khó thay đổi). Nó nên chứa abstraction, để concrete implementation ở volatile components.

Đo abstractness: ratio $A = \text{abstract_classes} / \text{total_classes}$.

Main Sequence

SDP + SAP cho relationship I vs A:

- Pain zone: $I = 0$, $A = 0$, stable concrete (rigid, khó thay đổi).
- Useless zone: $I = 1$, $A = 1$, unstable abstract (no purpose).
- Main sequence: $I + A = 1$, ideal line.



Component nên gần Main Sequence. Far from it = warning sign.

Sai lầm thường gặp

Sai lầm 1: Component quá nhỏ

Tách thành 100 component cho project medium. Build time chậm. Dependency hell. Coordination cost cao.

Quy tắc: component count khoảng = team count * 2-5. Vd: team 5 $\square$ 10-25 components.

Sai lầm 2: Component quá lớn

Tách quá lười. Component to $\square$ khó test, khó deploy parts riêng, vi phạm CCP.

Quy tắc: component khoảng 5-50 class (cho mediocre complexity).

Sai lầm 3: Cyclic dependencies tolerated

“Tạm thời” cycle tồn tại nhiều năm. Refactor khó hơn theo thời gian. Mỗi build slow vì cycle force rebuild widely.

Quy tắc: zero tolerance cho cycle ở component level. Detect + fix immediately.

Sai lầm 4: Component theo layer (horizontal slicing)

Đã nói ở 3.4. Component-by-layer = distributed monolith. Component nên by domain (vertical).

Tóm tắt

- **Component** = independently deployable unit, lớn hơn class.
- **Identify**: entity-based hoặc workflow-based (thường mix).

- **Cohesion principles:** REP (release together), CCP (change together), CRP (use together), trade-off.
- **Coupling principles:** ADP (no cycle), SDP (depend toward stable), SAP (stable = abstract).
- **Main Sequence:** $I + A = 1$.

Tổng kết Phần 4

Hết Phần 4. Tóm tắt:

Bài	Insight chính
4.2 FR vs NFR	Architecture driven bởi NFR, không FR
4.3 Identifying	Top 5-7 QA, measurable, prioritize qua workshop
4.4 Component thinking	REP/CCP/CRP cohesion + ADP/SDP/SAP coupling, Main Sequence

Phần 5-6 (Architecture Styles) sẽ áp dụng tất cả: mỗi style là một cách compose component để tối ưu một bộ QA cụ thể.

Bài tiếp: [Tổng quan Phần 5 - Fundamental Architecture Styles](#).